03 - Strings y clases envoltorio

Pregunta número 1

Which of the following lines of code correctly appends a string to a StringBuilder object in Java?

Seleccione una:

- a. StringBuilder sb = new StringBuilder("Hello"); sb.add(" World");
- b. StringBuilder sb = new StringBuilder("Hello"); sb.append(" World");
- c. StringBuilder sb = "Hello"; sb += " World";
- d. StringBuilder sb = new StringBuilder("Hello"); sb.insert(" World");

Respuesta: Lo primero que tenemos que recordar es que en Java, la clase StringBuilder se utiliza para crear cadenas de caracteres mutables. Para agregar texto al final de un objeto StringBuilder, se utiliza el método append(). La respuesta correcta es el inciso b por dos motivos cruciales. El primero es por la instanciación correcta: StringBuilder sb = new StringBuilder("Hello"); crea un nuevo objeto StringBuilder inicializado con el texto "Hello". Lo segundo es por el método correcto: sb.append(" World"); llama al método append, que añade la cadena " World" al final del contenido actual. El resultado sería "Hello World". Puedo decir que el punto clave de todo es la sintaxis correcta y siempre hay que tener presente que el método estándar para concatenar texto al final de un objeto StringBuilder existente es append().

Pregunta número 2

¿Cuál de las siguientes líneas de código utiliza correctamente un método de la clase String para reemplazar todas las apariciones del carácter 'a' por 'e' en una cadena dada en Java?

Seleccione una:

- a. String result = "banana".replace('a' ,'e');
- b. String result = "banana".change('a' ,'e');
- c. String result ="banana".modify('a' ,'e');
- d. String result = "banana".substitute('a' ,'e');
- e. String result = "banana".swap('a' ,'e');

Respuesta: La opción correcta es el inciso a, ya que es el que utiliza el método replace(char oldChar, char newChar), el cual es el método real y estándar de Java diseñado para sustituir todas las apariciones de un carácter específico por otro.

Dado que los objetos String son inmutables, este método no modifica la cadena original, sino que devuelve una nueva cadena resultante. Por otro lado, las opciones b, c, d y e utilizan nombres de métodos inventados (change, modify, substitute, swap). Ninguno de estos métodos existe en la definición de la clase String de Java, por lo que cualquiera de estas líneas resultaría en un error de compilación del tipo "cannot find symbol".

Pregunta número 3

What is the best way to test that the values of h1 and h2 are the same?

```
public static void main(String[] args){
    String h1 = "Bob";
    String h2 = new String("Bob");
}
```

Seleccione una:

- (●) a. if (h1.equals(h2)).
- ( ) b. if (h1 == h2).
- ( ) c. if (h1.toString() == h2.toString()).
- ( ) d. if (h1.same(h2)).

Respuesta: Aquí debemos contemplar la diferencia entre la igualdad de referencia y la igualdad de valor al trabajar con objetos String. La variable h1 se inicializa mediante un literal de cadena ("Bob"), por lo que Java la almacena en una memoria especial llamada String Pool. La variable h2 se instancia utilizando la palabra clave new String("Bob"). Esto obliga a la JVM a crear un objeto completamente nuevo en la memoria heap general, el cual es independiente del que está en el String Pool, aunque contengan el mismo texto. Por lo tanto, el inciso correcto es el a, ya que es la forma correcta porque el método .equals() está sobreescrito en la clase String para comparar lógicamente la secuencia de caracteres (el valor). Como ambas cadenas contienen "Bob", evaluará a true.

Pregunta número 4

What is the result?

```
public class X {
    public static void main(String[] args) {
        String theString = "Hello World";
        System.out.println(theString.charAt(11));
    }
}
```

Seleccione una:

- a. There is no output.
- b. d is output.
- c. A StringIndexOutOfBoundsException is thrown at runtime.
- d. An ArrayIndexOutOfBoundsException is thrown at runtime.
- e. A NullPointException is thrown at runtime.
- f. A StringArrayIndexOutOfBoundsException is thrown at runtime.

Respuesta: En este ejercicio se está evaluando el conocimiento sobre los límites de los índices en las cadenas de texto y las excepciones específicas que lanza la API de Java. Recordemos que en Java, los índices de un objeto String están basados en cero, lo que significa que el primer carácter comienza en la posición 0. La variable theString contiene el valor "Hello World". Si contamos los caracteres, incluyendo el espacio en blanco, la longitud total de la cadena es de 11 caracteres. Dado que la longitud es 11 y el índice comienza en 0, los índices válidos para esta cadena van del 0 (la letra 'H') al 10 (la letra 'd'). La instrucción theString.charAt(11) intenta acceder a un índice que excede el límite máximo permitido. Como resultado, la ejecución falla en tiempo de ejecución lanzando explícitamente una StringIndexOutOfBoundsException. Es fundamental distinguir esta excepción de la ArrayIndexOutOfBoundsException (opción d), ya que, aunque un String se respalda internamente en un arreglo, los métodos de su API lanzan la excepción específica de cadena cuando se brinda un índice inválido.

Pregunta número 5

What is the result?

```
public class Test {
    public void op2(String s) {
        s.toUpperCase();
    }
    public void op3() {
        String msg = new String("message");
        op2(msg);
        System.out.println(msg);
    }
    public static void main(String[] args) {
        new Test().op3();
    }
}
```

Seleccione una:

- a. MESSAGE
- b. An exception is thrown at runtime.
- c. Compilation fails.
- d. message
- e. null

Respuesta: El concepto que se está evaluando en este ejercicio es la inmutabilidad de los objetos de la clase String. En el flujo de ejecución podemos ver claramente este principio en acción. Primero tenemos el método op3; se instancia la variable msg apuntando al valor original "message" en minúsculas. Se invoca al método op2 pasando esta referencia. Dentro de op2, se ejecuta la instrucción s.toUpperCase(). Es fundamental recordar que los métodos de la clase String jamás alteran el objeto original; en su lugar, crean y devuelven un nuevo objeto en memoria. Dado que el código no captura ni reasigna ese valor devuelto (la línea no es s = s.toUpperCase();), el nuevo objeto "MESSAGE" queda solo en memoria y el objeto original referenciado por s permanece completamente inalterado. Al regresar el control al método op3, la instrucción System.out.println(msg); simplemente imprime el valor del objeto intacto; por lo tanto, se imprime “message”.

Pregunta número 6

What is the result?

```
public class Test {
    public static void main(String[] args) {
        String a = "tic";
        String b = "tac";
        a.substring(0, 2);
        a.concat("toc");
        System.out.println(a += "game" + b.toUpperCase());
    }
}
```

Seleccione una:

- a. ticgameTAC
- b. titocTAC
- c. tictactoc
- d. ticgametac
- e. Compilation fails.

Respuesta: Lo primero es recordar una regla fundamental en Java: los objetos de la clase String son inmutables. Esto significa que ningún método de la clase String puede cambiar el contenido del objeto original; en su lugar, los métodos como substring() o concat() devuelven un nuevo objeto String. Hay que ir paso a paso por el código. Tenemos las declaraciones: String a = "tic"; y String b = "tac". Tenemos la llamada a a.substring(0, 2). Este método extrae los caracteres desde el índice 0 hasta el 1 (el 2 es exclusivo), creando un nuevo String "ti". Sin embargo, el resultado no se guarda en ninguna variable. El objeto a sigue siendo "tic". Esta línea no tiene ningún efecto útil sobre a. Seguimos con la llamada a a.concat("toc"). Este método crea un nuevo String concatenando "tic" con "toc", resultando en "tictoc". Al igual que antes, el resultado no se asigna; por lo tanto, a sigue siendo "tic". Esto nos da otra línea que tampoco tiene efecto sobre a. De ahí seguimos con la línea de impresión: System.out.println(a += "game" + b.toUpperCase()). Primero, se evalúa b.toUpperCase(). Como b es "tac", el método devuelve un nuevo String "TAC". Luego, se evalúa la parte derecha de la asignación: "game" + "TAC" da como resultado "gameTAC". Después, se ejecuta el operador +=. Esto es un atajo para a = a + "gameTAC". Sustituyendo el valor actual de a (que sigue siendo "tic"),

tenemos: a = "tic" + "gameTAC". El resultado de esta concatenación es "ticgameTAC". Finalmente, System.out.println imprime el nuevo valor de a. Entonces la salida final es ticgameTAC.

Pregunta número 7

What will be the output of the following code snippet?

```
public class StringConcatenationTest {
    public static void main(String[] args) {
        String str1 = "Hello";
        String str2 = "World";
        String str3 = str1 + " " + str2;
        String str4 = str1.concat(" ").concat(str2);
        String str5 = new StringBuilder().append(str1).append(" ").append(str2).toString();
        System.out.print(str3.equals(str4) + " ");
        System.out.print(str4.equals(str5) + " ");
        System.out.print((str3 == str4) + " ");
        System.out.println(str4 == str5);
    }
}
```

Seleccione una:

- a. true true true true
- b. Compilation fails.
- c. true true false false
- d. false false false false
- e. false true false true
- f. false false true true

Respuesta: En este ejercicio estamos combinando la concatenación de cadenas con la evaluación de igualdad de valor (equals()) versus la igualdad de referencia (==). Hay que analizar cómo Java maneja cada variable en memoria. Comenzamos con la creación de objetos en tiempo de ejecución: Las variables str3, str4 y str5 se construyen dinámicamente durante la ejecución del programa. str3 usa el operador de concatenación con variables, str4 encadena métodos concat(), y str5 utiliza StringBuilder. La regla de oro aquí es que cualquier concatenación o manipulación de cadenas que involucre variables o métodos genera un objeto String completamente nuevo en la memoria heap general, no en el String Pool. Continuamos con la igualdad de valor (equals): Las dos primeras instrucciones imprimen el resultado de str3.equals(str4) y str4.equals(str5). Como el método .equals() compara la secuencia real de caracteres y las tres variables contienen exactamente el texto "Hello World", ambas evaluaciones dan como resultado true. Nos pasamos a la igualdad de referencia (==): Las dos últimas instrucciones imprimen el resultado de str3 == str4 y str4 == str5. El operador == verifica si las

variables apuntan exactamente al mismo objeto en la memoria. Como establecimos que son tres objetos distintos creados de forma independiente en el heap, ambas comparaciones dan como resultado false. Teniendo como salida final que el programa imprime los siguientes resultados separados por espacios: true true false false.

Pregunta número 8

Que se imprime en pantalla al ejecutar el siguiente código?

```
public class StringInternExample {
    public static void main(String[] args) {
        String str1 = new String("example");
        String str2 = "example";
        String str3 = str1.intern();
        System.out.println(str1 == str2);
        System.out.println(str2 == str3);
        System.out.println(str1 == str3);
    }
}
```

Seleccione una:

- (●) a. false true false
- ( ) b. true true true
- ( ) c. false false false
- ( ) d. true false false

Respuesta: En este ejercicio se está evaluando el conocimiento sobre la gestión de memoria de Java, específicamente el "String Pool" y el comportamiento del método intern(). Tenemos primero el String str1 = new String("example"): Al utilizar la palabra clave new, obligamos a la JVM a crear un objeto String completamente nuevo en la memoria heap general, ignorando el String Pool para esa referencia. Luego en String str2 = "example": Al inicializarse mediante un literal, la JVM coloca (o recupera) este objeto directamente desde la memoria especial del String Pool. Mientras que en String str3 = str1.intern(): El método intern() toma el valor de str1 ("example") y busca si ya existe una cadena idéntica en el String Pool. Como sí existe (la que referenció str2), intern() devuelve esa referencia exacta del Pool. Por

lo tanto, str3 ahora apunta al mismo objeto en el String Pool que str2. Evaluando las impresiones (== compara referencias de memoria): str1 == str2 es false porque str1 está en el heap general y str2 está en el String Pool. Pasando a str2 == str3, tenemos que es true porque ambas variables apuntan exactamente al mismo objeto dentro del String Pool. Finalmente, str1 == str3 es false porque str1 sigue apuntando al objeto en el heap general, mientras que str3 apunta al objeto en el String Pool. Con esto la salida nos da false true false.

Pregunta número 9

Given the following code snippet, what will be the output of the program?

```
String s1 = "text";
String s2 = new String("text");
String s3 = "text";
String s4 = s2.intern();
System.out.println(s1 == s3);
System.out.println(s1 == s2);
System.out.println(s1 == s4);
```

Seleccione una:

- a. false false true
- b. true true true
- c. true false false
- d. true false true

Respuesta: En este ejercicio continuamos evaluando el dominio que tenemos sobre el "String Pool" y las referencias en memoria. Hay que ir desglosando cada variable para entender qué está pasando. Primero vamos con String s1 = "text": La JVM coloca el literal "text" en el String Pool y hace que s1 apunte a él. Seguimos con String s2 = new String("text"): El uso de new fuerza la creación de un objeto completamente nuevo en la memoria heap general, independiente del que está en el String Pool. Continuamos con String s3 = "text": Al usar un literal que ya existe, la JVM reutiliza el objeto del String Pool creado en el paso 1. Por lo tanto, s1 y s3 apuntan exactamente a la misma dirección de memoria. En String s4 = s2.intern(): El método intern() busca el valor de s2 ("text") dentro del String Pool. Como el valor ya existe (es el objeto al que apuntan s1 y s3), el método devuelve la referencia de ese objeto del pool. Ahora s4 también apunta al mismo objeto que s1 y s3. Evaluando las comparaciones con el operador == (que verifica igualdad de referencias en memoria), tenemos que s1 == s3 es true porque ambas apuntan al mismo objeto en el String Pool. Por otro lado, s1 == s2 es false porque s1 está en el

Pool y s2 es un objeto distinto en el heap general. Finalmente, s1 == s4 es true porque intern() devolvió la referencia del objeto que ya estaba en el Pool, haciendo que s4 apunte al mismo lugar que s1. Con lo anterior dicho, tenemos que la salida final es true false true.

Pregunta número 10

¿Qué imprime en pantalla este código?

```
public class Test {
    public static void main(String[] args) {
        String str = "a1b2c3d4e5f6";
        String[] parts = str.split("\\D");
        for (String part : parts) {
            System.out.print(part + " ");
        }
    }
}
```

Seleccione una:

- a. 1 2 3 4 5 6
- b. Compilation fails.
- c. abcdef
- d. An exception is thrown at runtime.
- e. 123456

Respuesta: En este código se está evaluando el comportamiento del método split() de la clase String cuando se utiliza en combinación con expresiones regulares (regex). Tenemos que desglosar la ejecución paso a paso para entender el código. Lo primero es la definición de la expresión regular: La instrucción str.split("\\D") utiliza el metacarácter \D (que debe escaparse en Java con doble barra invertida \\D). Este metacarácter representa cualquier carácter que no sea un dígito numérico. Por lo tanto, actuará sobre las letras 'a', 'b', 'c', 'd', 'e' y 'f'. Continuamos con la división de la cadena: El método split utiliza esas letras como "cortes" o delimitadores en la cadena "a1b2c3d4e5f6". Lo siguiente es el manejo de delimitadores iniciales: Un detalle técnico crucial de la API es que, como la cadena comienza directamente con un delimitador (la letra 'a'), el método genera una cadena vacía "" como el primer elemento del arreglo resultante. Por lo tanto, el

arreglo parts queda estructurado en memoria como: ["", "1", "2", "3", "4", "5", "6"]. Terminamos con la iteración e impresión: El ciclo for-each recorre este arreglo. En cada iteración, imprime el elemento actual concatenado con un espacio en blanco mediante System.out.print(part + " "). La primera iteración imprime la cadena vacía más un espacio (" "). Las iteraciones subsecuentes imprimen "1 ", "2 ", "3 ", etc. La salida generada en la consola será una secuencia de los números extraídos, separados por espacios. Es importante recalcar que la salida real tendría un espacio al principio (1 2 3 4 5 6). Sin embargo, el inciso a es el que refleja la intención del ejercicio.

Pregunta número 11

What is the result?

```
1 public class Boxer1{
2     Integer i;
3     int x;
4     public Boxer1(int y){
5         x = i+y;
6         System.out.println(x);
7     }
8     public static void main(String[] args){
9         new Boxer1(new Integer(4));
10    }
11 }
```

Seleccione una:

- a. Compilation fails because of an error in line 5
- b. The value "4" is printed at the command line.
- c. A NullPointerException occurs at runtime
- d. An IllegalStateException occurs at runtime.
- e. Compilation fails because of an error in line 9
- f. A NumberFormatException occurs at runtime

Respuesta: Este es un ejercicio muy interesante, ya que el código compila sin problemas, pero falla en tiempo de ejecución al intentar realizar una operación aritmética con un objeto que no ha sido inicializado. Hay que ir paso a paso para

entender qué está sucediendo. Empezamos con la declaración de variables de instancia en las líneas 2 y 3: Integer i es una variable de tipo objeto (envoltorio). Su valor por defecto en Java es null, ya que no se le asigna ningún valor explícitamente en la clase. int x es una variable de tipo primitivo. Su valor por defecto es 0. Continuamos con la creación del objeto en la línea 9: new Boxer1(new Integer(4)); crea una instancia de la clase. El argumento new Integer(4) se pasa al constructor. Java realiza un unboxing automático para convertir ese objeto Integer al tipo primitivo `int y` que espera el constructor. La variable y recibe el valor 4. Nos llega un error en el constructor (línea 5): La línea x = i + y; intenta sumar i (que es null) y y (que es 4). Para poder sumar, Java necesita convertir el objeto Integer i a un tipo primitivo int (proceso llamado unboxing). Intenta ejecutar algo equivalente a i.intValue(). Como i es null, al intentar invocar el método intValue() sobre una referencia nula, la Máquina Virtual de Java (JVM) lanza una NullPointerException. Como consecuencia, el programa se detiene abruptamente en este punto y nunca llega a ejecutar la línea 6 (System.out.println(x);). El punto clave aquí es que, como i es nulo, invocar un método sobre una referencia nula desencadena inmediatamente una excepción NullPointerException en tiempo de ejecución.

Pregunta número 12

What is the result?

```
public class Test {
    public static void main(String[] args) {
        String str = "Hello";
        modify(str);
        System.out.println(str);
        StringBuffer sbf = new StringBuffer("Hello");
        modify(sbf);
        System.out.println(sbf.replace(2, sbf.length(), "o"));
        StringBuilder sbd = new StringBuilder("Hello");
        modify(sbd);
        System.out.println(sbd.reverse());
    }
    private static void modify(String s) {
        s = s.concat(" World");
    }
    private static void modify(StringBuffer sb) {
        sb.append(" World");
    }
    private static void modify(StringBuilder sb) {
        sb.append(" World");
    }
}
```

Seleccione una:

- a. Hello Hello World Hello World
- b. Hello Hello Hello
- c. Hello Heo dlroW olleH
- d. Helio World Heo Hello World
- e. Hello Heo World Hello

Respuesta: En este ejercicio se está evaluando la diferencia entre la inmutabilidad de String y la mutabilidad de StringBuffer y StringBuilder, así como el paso de parámetros por valor en Java. Vamos a ir paso a paso por el método main para poder entender el código. Primero tenemos el String str = "Hello"; y modify(str). Debemos tener presente que en Java, los objetos String son inmutables. Además, los parámetros se pasan por valor (se pasa una copia de la referencia). Dentro del método modify(String s), la línea s = s.concat(" World"); crea un nuevo objeto String y reasigna la variable local s para que apunte a él. La variable original str en el main sigue apuntando al objeto original "Hello". Por lo que System.out.println(str); imprime: Hello. En StringBuffer sbf = new StringBuffer("Hello"); y modify(sbf), StringBuffer es mutable. Cuando se pasa al método modify, se pasa una copia de la referencia, pero ambos apuntan al mismo objeto en memoria. La línea sb.append(" World"); modifica el contenido del objeto original. Ahora sbf contiene "Hello World". En System.out.println(sbf.replace(2, sbf.length(), "o")): sbf es "Hello World" (longitud 11), mientras que replace(2, 11, "o") reemplaza los caracteres desde el índice 2 hasta el final (índice 11) con la letra "o". Los caracteres en los índices 0 y 1 son 'H' y 'e'. El resto se reemplaza por 'o'. El resultado es "He" + "o" = "Heo". Se imprime Heo. En StringBuilder sbd = new StringBuilder("Hello"); y modify(sbd); StringBuilder también es mutable. Al igual que con StringBuffer, el método append modifica el objeto original. Después de modify(sbd), el contenido de sbd es "Hello World". System.out.println(sbd.reverse()); invierte la secuencia de caracteres. "Hello World" invertido es "dlroW olleH". Se imprime: dlroW olleH. Con lo anterior ya plasmado, tenemos que la salida final al concatenar las tres impresiones (Hello, Heo, dlroW olleH) es Hello Heo dlroW olleH.